

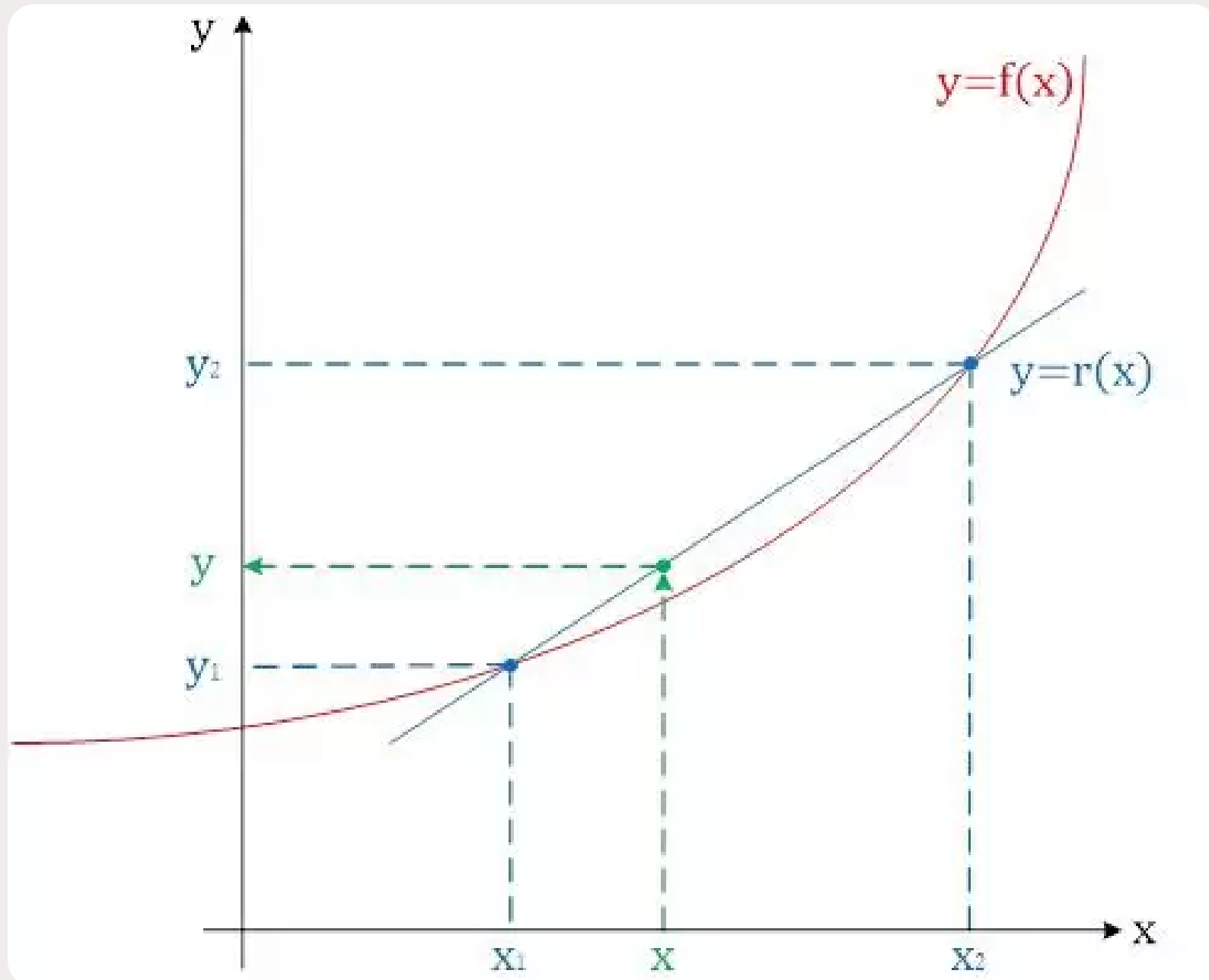


Interpolación Cuadrática

Alondra Joseline Gaucín Ramírez
Alexis Ramsés Castillo Abraham
Matías Loyola Velázquez



Qué es la interpolación



Interpolación es el método mediante el cual, a partir de un conjunto de datos discretos, se construye una función que pasa exactamente por esos puntos y permite estimar valores intermedios.

Se parte de mediciones o valores conocidos en ciertos puntos y se asume que existe un comportamiento continuo entre ellos. La interpolación consiste en reconstruir ese comportamiento mediante una función, que cumple las condiciones impuestas por los datos.

Interpolación cuadrática

- Método que aproxima una función usando una parábola que pasa exactamente por tres puntos dados.
- Se hace con un polinomio de segundo grado, por lo que se utiliza una función cuadrática (parábola).
- La interpolación exige que el polinomio que une los puntos sea único y de menor grado, entonces se requieren 3 puntos para cumplir esta restricción.

Forma general:

$$f(x) = a + bx + cx^2$$

Forma matemática

Interpolación Cuadrática

$$f(x) = y_0 + \frac{(y_1 - y_0)}{(x_1 - x_0)}(x - x_0) + \frac{\frac{(y_2 - y_1)}{(x_2 - x_1)} - \frac{(y_1 - y_0)}{(x_1 - x_0)}}{(x_2 - x_0)}(x - x_0)(x - x_1), x \in [x_0, x_2]$$

Construye una parábola que pasa por tres puntos conocidos: (x_0, y_0) , (x_1, y_1) , (x_2, y_2)

Donde:

$$f(x_0) = y_0$$

$$f(x_1) = y_1$$

$$f(x_2) = y_2$$

Interpretación por partes

Primer término:

y_0

Valor inicial y
valor conocido

Segundo término:

$$\frac{(y_1 - y_0)}{(x_1 - x_0)} (x - x_0)$$

Interpolación lineal. Es
una recta y representa
la pendiente entre dos
puntos

Tercer término:

$$\frac{\frac{(y_2 - y_1)}{(x_2 - x_1)} - \frac{(y_1 - y_0)}{(x_1 - x_0)}}{(x_2 - x_0)} (x - x_0)(x - x_1)$$

Corrección cuadrática.
Detecta la curvatura y
convierte la recta en
parábola

Fundamentos

Unicidad del polinomio

Principio matemático clave:
Para un conjunto de $n+1$ puntos, existe un único polinomio de grado n que pasa por todos.

Forma de Lagrange

Usa una combinación lineal de polinomios base, para el caso cuadrático:

$$y(x) = \frac{(x - x_2)(x - x_3)}{(x_1 - x_2)(x_1 - x_3)}y_1 + \frac{(x - x_1)(x - x_3)}{(x_2 - x_1)(x_2 - x_3)}y_2 + \frac{(x - x_1)(x - x_2)}{(x_3 - x_1)(x_3 - x_2)}y_3$$

Error de interpolación

El error de un polinomio cuadrático depende de la tercera derivada de la función original (f''') y de la distancia entre los puntos.

$$f(x) - P(x) = \frac{f^{(3)}(\xi)}{3!}(x - x_0)(x - x_1)(x - x_2)$$

Forma de Newton (Diferencias Divididas)

La forma de Newton permite construir el polinomio de forma incremental

$$P(x) = y_0 + (x - x_0)f[x_0, x_1] + (x - x_0)(x - x_1)f[x_0, x_1, x_2]$$

Donde $f[-]$ son las diferencias divididas.

Uso en Python

¿Cuál es la librería?

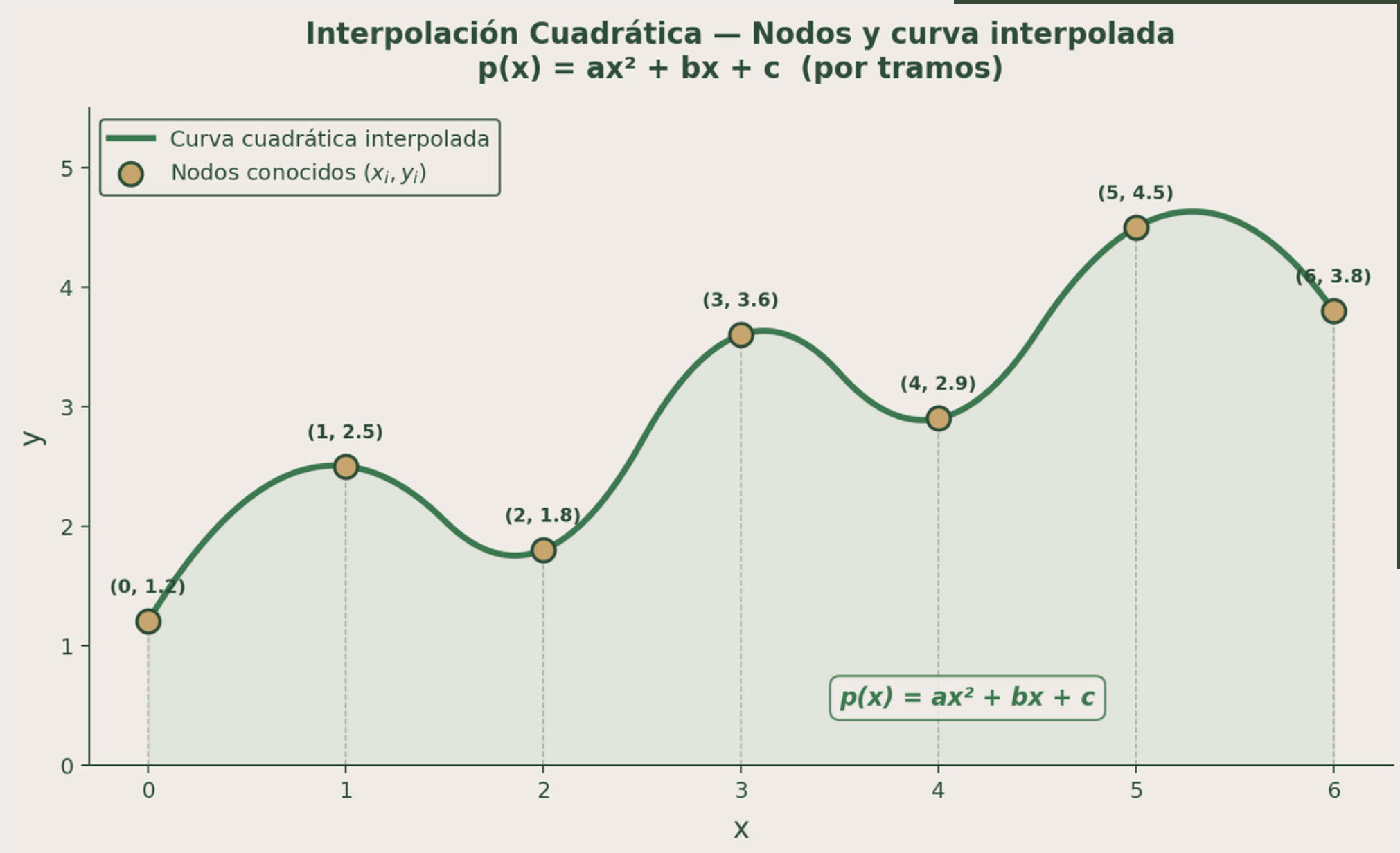
SciPy (Scientific Python) incluye interpolación cuadrática a través del módulo `scipy.interpolate`, específicamente con la función `interp1d(kind='quadratic')`. Es una librería de código abierto, construida sobre NumPy, estándar en el ecosistema científico de Python. Se instala con `pip install scipy`.



Uso en Python

¿Cómo resuelve?

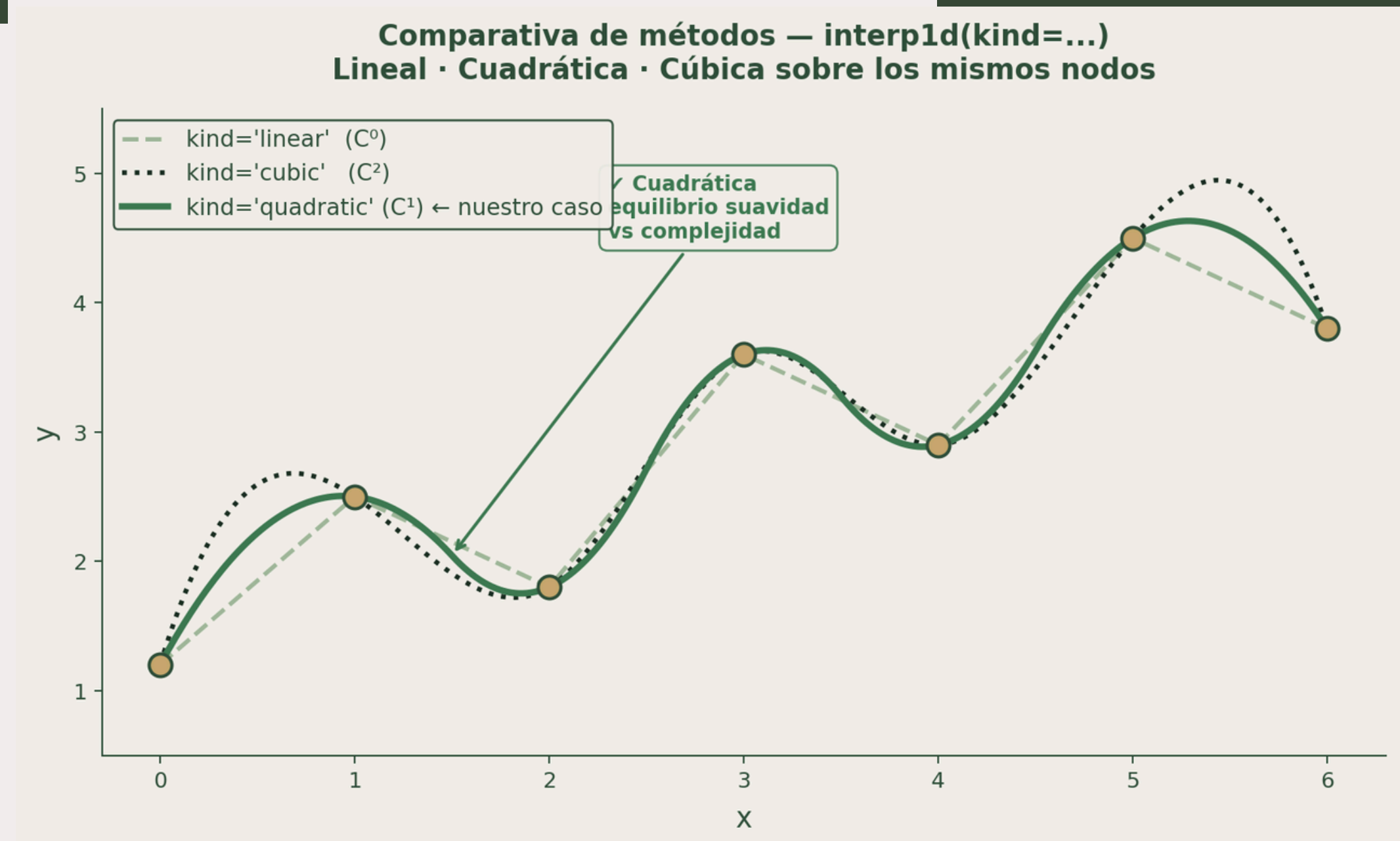
interp1d recibe los arreglos de nodos conocidos x e y , y construye un polinomio local de grado 2 — de la forma $p(x) = ax^2 + bx + c$ — para cada tramo entre nodos consecutivos. Impone condiciones de continuidad en los extremos de cada tramo (valores y primeras derivadas coinciden), y devuelve una función evaluable $f(x_nuevo)$ que localiza el tramo correcto y evalúa el polinomio local correspondiente.



Uso en Python

Detalles técnicos principales:

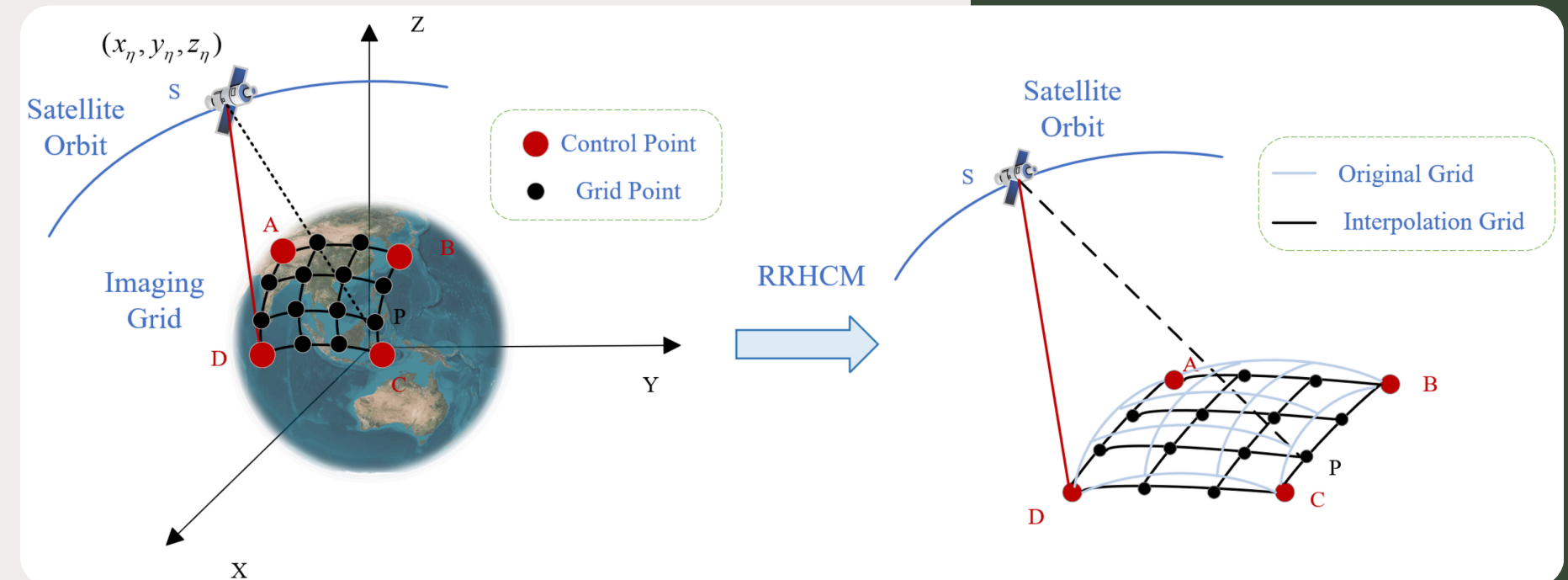
- Requiere mínimo 3 nodos de datos
- Interpolación por piezas (piecewise quadratic spline), continuidad C^1
- Parámetros clave: `kind='quadratic'`, `bounds_error`, `fill_value`
- No extrapola por defecto fuera del rango de los datos
- Alternativa moderna en SciPy ≥ 1.11 :
`make_interp_spline(x, y, k=2)`



Aplicaciones

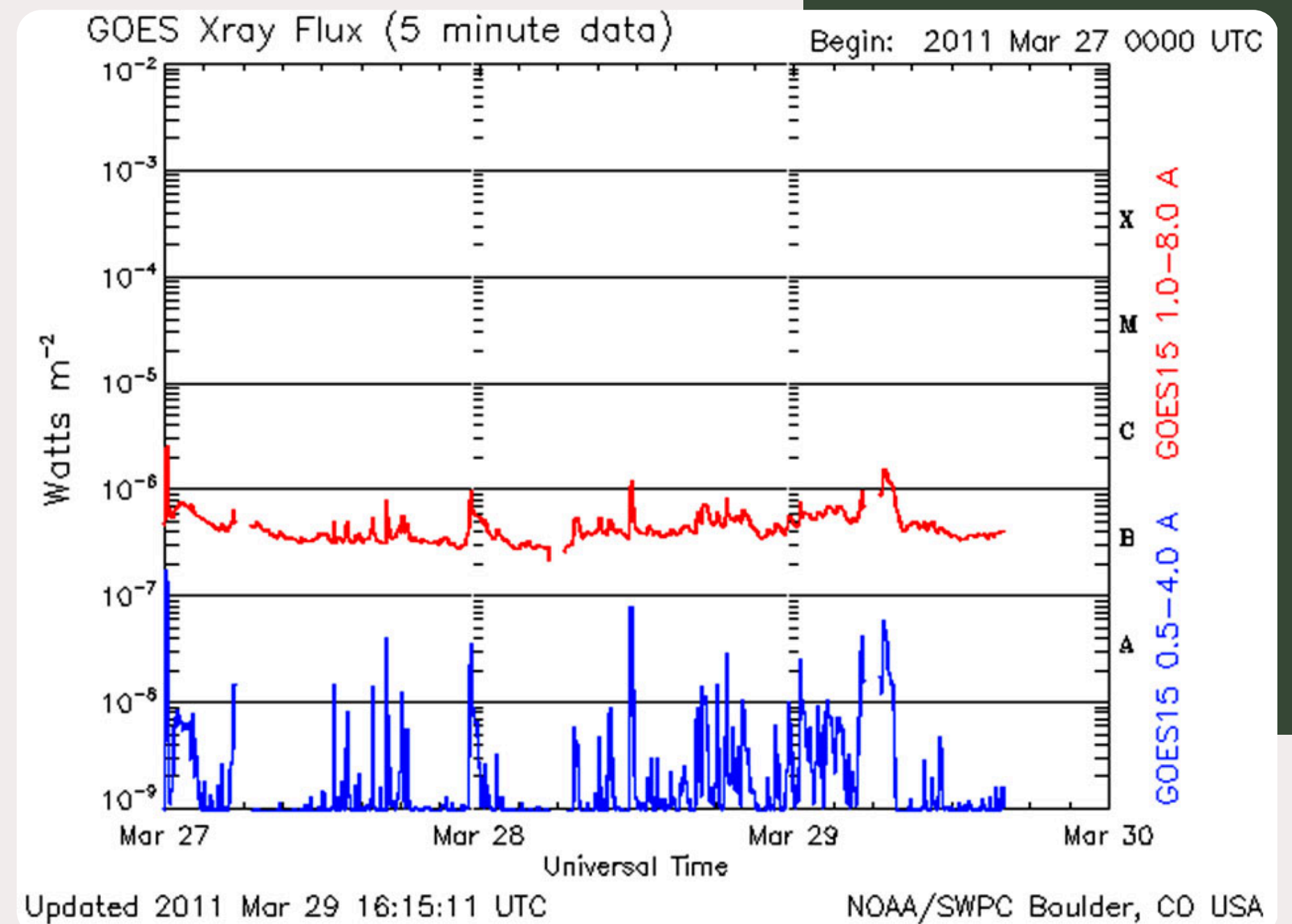
Reconstrucción de trayectorias orbitales a partir de posiciones discretas, para estimar estados intermedios en navegación espacial.

Estimación de velocidad y aceleración de satélites usando interpolación local sobre datos de telemetría.



Estimación del tiempo exacto de picos en emisiones solares dentro del clima espacial, cuando el muestreo es discreto.

Interpolación de campos físicos (gravitacionales o radiativos) a partir de datos puntuales en simulaciones.



Código Ejemplo

Librerías

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d
```

Datos

```
t = np.array([0, 1, 2, 3, 4])
I = np.array([1.0, 2.5, 4.2, 3.8, 2.0])
```

Elección de 3 puntos

```
t_local = t[1:4]
I_local = I[1:4]
```

Interpolación

```
# Interpolación cuadrática
f = interp1d(t_local, I_local, kind='quadratic')

# Evaluación en puntos
t_fino = np.linspace(t_local[0], t_local[-1], 200)
I_fino = f(t_fino)

# Encontrar el máximo
indice_max = np.argmax(I_fino)
t_max = t_fino[indice_max]
I_max = I_fino[indice_max]

# Gráfica
plt.scatter(t, I, label="Datos medidos")
plt.plot(t_fino, I_fino, label="Interpolación cuadrática")
plt.scatter(t_max, I_max, color='red', label="Pico estimado")

plt.xlabel("Tiempo")
plt.ylabel("Intensidad")
plt.legend()
plt.title("Interpolación cuadrática para estimar pico")
plt.show()

print(f"Tiempo del pico estimado: {t_max:.3f}")
print(f"Valor del pico estimado: {I_max:.3f}")
```

Bibliografía

- Burden, R. L., & Douglas Faires, J. (s. f.). Análisis numérico (7.a ed.).
- Jiménez Bedolla, J. C. (2022). Métodos numéricos usando Python con aplicaciones a la Ingeniería Química. En Universidad Nacional Autónoma de México-Facultad de Química.
<https://librosoa.unam.mx/bitstream/handle/123456789/3416/MNPython.pdf?sequence=1&isAllowed=y>
- interp1d — SciPy v1.17.0 Manual. (s. f.).
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.interp1d.html>
- make_interp_spline — SciPy v1.17.0 Manual. (s. f.).
https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.make_interp_spline.html
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., Van Der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., . . . Vázquez-Baeza, Y. (2020a). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261-272. <https://doi.org/10.1038/s41592-019-0686-2>
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., Van Der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., . . . Vázquez-Baeza, Y. (2020b). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261-272. <https://doi.org/10.1038/s41592-019-0686-2>
- Yousif, H., & El-Rabbany, A. (2007). Assessment of Several Interpolation Methods for Precise GPS Orbit. *Journal Of Navigation*, 60(3), 443-455. <https://doi.org/10.1017/s0373463307004250>